Appunti del corso di
Basi di dati

UniVR - Dipartimento di Informatica

primo semestre - A.A. 2025/26

Mattia Nicolis Matteo Drago

Indice

1 Introduzione **1**
1.1 Nascita dei DBMS . . . 2
1.2 Architettura dei DBMS . . . 3

2 Progettazione di una base di dati **5**
2.1 Progettazione dei dati . . . 6
2.2 Requisiti . . . 6
2.2.1 Progettazione concettuale . . . 6
2.2.2 Progettazione logica . . . 6
2.2.3 Progettazione fisica . . . 6
2.3 Stategie di progettazione . . . 7

3 Progettazione concettuale **8**
3.1 Entità . . . 8
3.2 Relazione . . . 9
3.3 Attributo . . . 11
3.4 Identificatore . . . 12
3.5 Cardinalità . . . 13
3.6 Generalizzazione di entità . . . 14

4 Progettazione logica **16**
4.1 Progettazione nel modello relazionale . . . 19
4.1.1 Relazione unica . . . 20
4.1.2 Decomposizione della relazione unica . . . 21
4.2 Valori nulli . . . 23
4.3 Vincoli d'integrità . . . 24
4.3.1 Vincoli di dominio . . . 24
4.3.2 Vincoli di tupla . . . 24
4.3.3 Vincoli di intrarelazionali . . . 25
4.3.4 Vincoli interrazionali . . . 26
4.4 Progettazione logica dallo schema concettuale . . . 27
4.4.1 Ristrutturazione dello schema logico . . . 27
4.4.1.1 Analisi delle ridondanze per la presenza di dati derivabili . . . 28
4.4.1.2 Eliminazione delle generalizzazioni . . . 28
4.4.1.3 Accorpamento (o partizionamento) di entità e relazioni . . . 29
4.4.1.4 Scelta degli identificatori principali . . . 29

4.4.2 Traduzione nel modello logico 30

5 Tecnologie NoSQL **31**
5.1 Sistemi basati su documenti (document store) 32
5.1.1 Caratteristiche principali 32
5.1.2 Progettazione dei dati come documenti 32
5.1.3 Traduzione da schema E-R a collezione di documenti 33
5.1.3.1 Prima fase: individuazione delle entità principali 33
5.1.3.2 Seconda fase: incapsulamento delle entità non principali . . . 34
5.1.3.3 Terza fase: rappresentazione delle relazioni non già rappresentate strutturalmente 37
5.1.3.4 Quarta fase: regole di traduzione di uno schema etichettato . 39

6 Operazioni su una base di dati relazionale **41**
6.1 Algebra relazionale 41
6.1.1 Operatori insiemistici 42
6.1.2 Operatori specificici 43
6.2 Calcolo relazionale 44

7 Esercizi **45**
7.1 Progettazione concettuale 45
7.1.1 Esercizio 1 - scheda 45
7.1.2 Esercizio 2 - scheda 46
7.1.3 Esercizio 3 - scheda 46
7.1.4 Esercizio 4 - scheda 46
7.2 Progettazione relazionale 46
7.2.1 46

1

Introduzione

La nascita dei sistemi per la gestione di basi di dati ha visto due momenti principali:

anni '60 : sviluppo di applicazioni negli **ambienti di ricerca scientifica**
anni '70 : sviluppo di applicazioni informatiche in **ambito gestionale**

Si trattava di semplici dispositivi in cui gli algoritmi di elaborazione erano semplici e grandi quantità di dati erano condivisi da più applicazioni.
Queste caratteristiche erano specifiche per l'ambiente in cui erano state introdotte ovvero il **sistema informativo**.

Definizione: sistema informativo

Un **sistema informativo** è l'insieme delle attività umane e dei dispositivi di memorizzazione ed elaborazione che organizza e gestisce l'informazione di interesse per un'organizzazione di dimensioni qualsiasi.

Il sistema informativo è costituito da **dati** e **informazioni**:

dato
elemento di conoscenza di base costituito da simboli che devono essere elaborati
↓
informazione
interpretazione dei dati che permette di ottenere conoscenza più o meno esatta di fatti e situazioni

Lo studio del sistema informativo avviene attraverso i **diagrammi di flusso**, i quali permettono di svolgere diverse operazioni tra cui:

- definizione di archivi di dati e delle sorgenti di dati
- definiazione degli utenti
- definizione di procedure e processi
- definizione dei flussi di dati

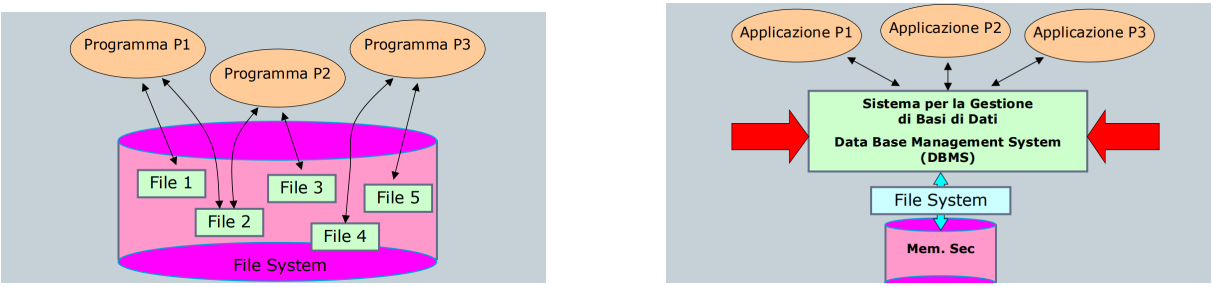
1.1 Nascita dei DBMS

Negli anni '70 i programmi comunicavano direttamente con i dati contenuti dei file system.

Si trattava di:

- ▷ **operazione scomoda**: l'accesso ai dati su file era scarso
- ▷ **rindonza nei dati**: duplicazione dello stato dato su più file
- ▷ **inconsistenza**: dovuta ad aggiornamenti parziali
- ▷ **progettazione dei dati era replicata per ogni programma**

Negli anni '80 la soluzione che venne proposta fu quella di inserire un **DBMS** (Data Base Management System) tra file system e applicazioni.



(a) file system **(b)** file system + base di dati

Figura 1.1: prima e dopo l'innovazione

Definizione: base di dati

Una **base di dati** è una collezione di dati dati utilizzati per rappresentare con la tecnologia informatica le informazioni di interesse per un sistema informativo.

Definizione: DBMS

Un **DBMS** è un sistema che gestisce su memoria secondaria collezioni di dati grandi, condivisi e persistenti, assicurandone l'**affidabiltà**, la **privatezza** e l'**accesso efficiente**.

Questo nuovo approcio ha portato a numerosi vantaggi:

⇒ **maggiore astrazione** e **potenza espressiva** per descrivere le proprietà del dato.
All'interno di un DBMS, i dati vengono interpretati come **oggetti** (istanze di classi o righr di tabelle).
Prima erano interpretati come blocchi o pagine di memoria secondaria (sequenze di byte).

⇒ **operazioni di asccesso di dati più complesse** basate su un linguaggio di interrogazione (*select from where*), anzichè attraverso operazioni di *read/write*

⇒ **migliore interazione uomo-informazione**:
  ⋆ **linguaggio per la definizione dei dati** → **D**ata **D**efinition **L**enguage

- ⋆ **linguaggio di manipolazione**: popola la base di dati e modifica il suo contenuto con aggiunte, cancellazioni e variazioni sui dati (es. SQL)

Definizione: modello di dati

Un **modello di dati** è l'insieme di costrutti forniti dal DBMS per descrivere la struttura e le proprietà dell'informazione contenuta in una base di dati.

I costrutti permettono di definire le strutture dati che conterranno le informazioni e specificare le proprietà che dovranno soddisfare le istanze.

Nel tempo si sono susseguiti diversi modelli di dati:

⇒ **nel passato**: gerarchico, reticolare

⇒ **attualmente**: relazionale, a oggetti, object-relational (SQL:1999), document-oriented (Json), NoSQL

Definizione: schema di una base di dati

Lo **schema di una base di dati** (invariante nel tempo) è la descrizione della struttura e delle proprietà di una specifica base di dati fatta utilizzando i costrutti del modello dei dati.

Definizione: istanza di una base di dati

L'**istanza di una base di dati** (variante nel tempo) è costituita dai valori che in un certo istante popolano le strutture dati della base di dati.

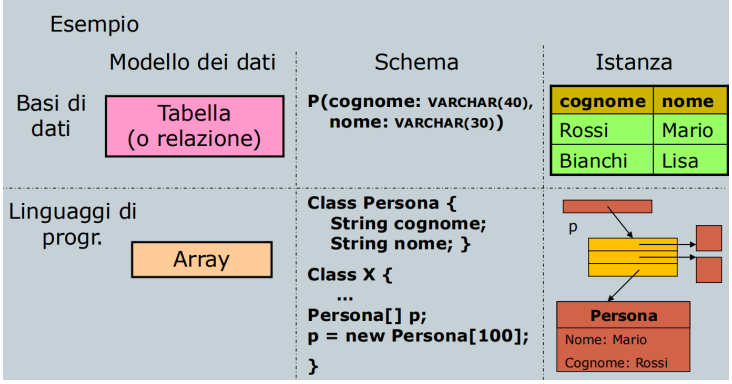


Figura 1.2: distinzione tra modello, schema e istanza

1.2 Architettura dei DBMS

Lo schema di una base di dati è diviso in tre livelli:

livello interno

rappresenta la base di dati per mezzo delle strutture fisiche di memorizzazione (file dati, file indice, ecc.)

↓

livello logico

rappresenta la struttura e le proprietà della base di dati definita attraverso i costrutti del modello dei dati del DBMS

↓

livello esterno

descrive una porzione dello schema logico di interesse per uno specifico utente o applicazione

La caratteristica fondamentale di queste strutture è l'**indipendenza**:

- ⋆ **fisica**: schema logico completamente indipendente dallo schema interno.
  Ne consegue che le variazioni delle strutture fisiche non impattano sullo schema logico e, quindi, sulle applicazioni
- ⋆ **logica**: schemi esterni indipendenti dallo schema logico.
  Ne consegue che le variazioni dello schema logico (senza rimuovere nessun dato) non impattano sugli schem esterni e, quindi, sulle applicazoni

2

Progettazione di una base di dati

La **progettazione di una base di dati** costituisce solo una delle componenti del processo di sviluppo di un sistema informativo e va, quindi, inquadrata in un contesto più ampio: **ciclo di vita** dei sistemi informativi.

Il ciclo di vita di un sistema informativo comprende:

studio della fattibilità
definisce i costi delle varie alternative possibili e stabilisce le proprietà di realizzazione delle varie componenti del sistema

↓

raccolta e analisi dei requisiti
individua e studia le proprietà e le funzionalità del sistema, producendo una descrizione completa, ma informale dei dati coinvolti e delle operazioni su di essi

↓

progettazione
→ *progettazione dei dati*: descrive la struttura e l'organizzazione dei dati
→ *progettazione delle applicazioni*: descrive in modo formale le caratteristiche dei programmi applicativi
Le due attività attività sono complementari e possono procedere in parallelo o a cascata

↓

implementazione
realizza il sistema informativo secondo la struttura e le caretteristiche definite nella fase di progettazione.
Viene costruita e popolata la base di dati e viene prodotto il codice dei programmi

↓

validazione e collaudo
verifica il corretto funzionamento e la qualità del sistema informativo.
La sperimentazione deve prevedere, per quanto possibile, tutte le condizioni operative

↓

funzionamento
entra in funzione il sistema informativo ed esegue i compiti per i quali era stato originariamente progettato.

Se non si verificano malfunzionamenti o revisioni delle funzionalità del sistema, questa attività richiede solo operazioni di gestione e manutenzione

2.1 Progettazione dei dati

Una metodologia di progettazione è costituita da:

- ▷ **decomposizione**: dividere in passi (indipendenti) le attività di progetto
- ▷ **strategie**: individuare un insieme di strategie e criteri di scelta da seguire
- ▷ **modelli di riferimento**: utilizzare modelli di dati e tecniche di progettazione consolidate

Una buona metodologia deve essere:

- **generale**: rispetto al problema studiato e agli strumenti utilizzati
- **prodotto di qualità**: in termini di correttezza, completezza e efficienza delle risorse impiegate
- **facile da usare**: nelle strategie e nei modelli di riferimento

2.2 Requisiti

Nell'ambito delle basi di dati si è consolidata negli anni una metodologia di progettazione che ha dato prova di soddisfare pienamente le proprietà descritte.
Tale metodologia è articolata in tre fasi principali da efettuare a cascata.

2.2.1 Progettazione concettuale

La **progettazione concettuale** è la prima fase della progettazione dei dati.
Lo scopo è quello di produrre una descrizione formale dei dati (schema concettuale).
Lo schema deve essere indipendente dall'implementazione.

Non è solo un progetto intermedio, ma costituisce anche una porzione del risultato finale perchè rappresenta una descrizione di alto livello del contenuto della base di dati, comprensibile anche per utenti poco esperti.

2.2.2 Progettazione logica

La **progettazione logica** è la seconda fase della progettazione dei dati.
Lo scopo è quello di tradurre lo schema concettuale in uno schema logico in modo da poterlo utilizzare su un sistema specifico.

Lo schema logico infatti è dipendente dalle tecnologie utilizzate e bisogna tenere anche in considerazione le operazioni più frequenti che le applicazioni effettueranno sulla base di dati.

2.2.3 Progettazione fisica

La **progettazione fisica** è la terza fase della progettazione dei dati.
Lo scopo è quello di ottimizzare l'accesso ai dati completando lo schema logico con i parametri relativi alla memorizzazione fisica dei dati e con gli opportuni metodi d'accesso (*indici*).

2.3 Stategie di progettazione

Lo sviluppo do uno schema concettuale può essere visto come un processo di ingegnerizzazione ed è, quindi, possibile applicare anche a tale processo le strategie classiche:

- **top-down** $\downarrow$: consiste nel considerare le specifiche globalmente e produrre uno schema iniziale completo ma con pochi concetti molto astratti.
  Si va poi a raffinare i concetti astratti fino ad arrivare allo schema concettuale completo in ogni dettaglio
- **bottom-up** $\uparrow$: consiste nel decomporre le specifiche iniziali in parti elementari e produrre uno schema iniziale dettagliato
- **inside-out**: consiste nell'individuare nelle specifiche alcuni concetti importanti (*concetti guida*) e partendo da quelli generare gli schemi per i relativi concetti.
  Si procede fondendo gli schemi precedenti per generare lo schema finale

3

Progettazione concettuale

Il **modello entità-relazione** (E-R) è un modello formale e non ambiguo, utilizzato per la progettazione concettuale di una base di dati.
Fornisce una serie di strutture (*costrutti*), con sintassi grafica, per specificare la struttura e le proprietà dei dati da rappresentare indipendentemente dalla tecnologia.

3.1 Entità

Definizione: entità

Un'**entità E** rappresenta una classe di oggetti con:

- ▷ **proprietà comuni**
- ▷ **esistenza autonoma**: rispetto ad altre classi di oggetti
- ▷ **indipendenza univoca**: esiste una chiara corrispondenza tra gli oggetti istanze di entità e concetti istanziati nel sistema informativo

Un entità si rappresenta con un rettangolo che contiene il nome dell'entità:

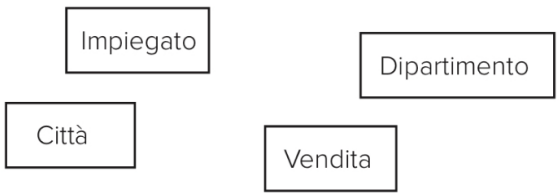


Figura 3.1: entità

Un'istanza dell'entità E è un oggetto appartenente alla classe rappresentata da E.
Si indica con $\mathbf{I(E)}$ l'insieme delle istanze di E che esistono nella base di dati in un certo istante e alla creazione della base di dati è vuota ($\mathrm{I(E)} = \emptyset$).

3.2 Relazione

Definizione: relazione

Una **relazione** rappresenta un legame logico tra due o più entità.
Può esserci anche una relazione all'entità stessa (relazione ricorsiva).

Una relazione si rappresenta nello schema con un rombo collegato con delle linee alle entità coinvolte nella relazione e a fianco il nome della relazione:

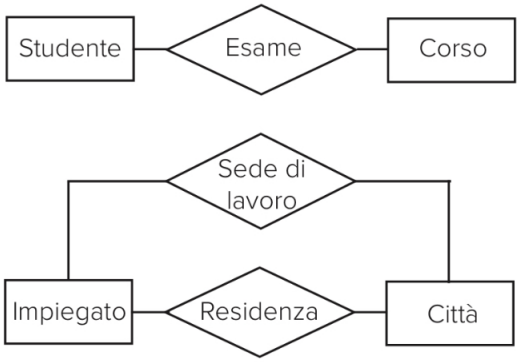


Figura 3.2: relazione tra entità

Data una relazione R tra n entità $E_1, \ldots, E_n$ un'istanza della relazione R è una ennupla di istanze di entità:

$$(e_1, \ldots, e_n) \text{ dove } e_i \in I(E_i) \text{ per } 1 \leq i \leq n$$

La popolazione di R rappresenta l'insieme delle coppie di istanze delle entità E e F che sono in relazione in un certo istante:

$$I(R) = \{(e_i, f_j) \mid I(E), f_j \in I(F)\}$$

Data una relazione R tra n entità $\{E_1, E_2, \ldots, E_n\}$ vale sempre la seguente proprietà sull'insieme delle istanze $I(R)$:

$$I(R) \subseteq I(E_1) \times I(E_2) \times \cdots \times I(E_n)$$

La conseguenza di questa proprietà è che non è possibilie rappresentare la stessa ennupla più volte.

Definizione: relazione ricorsiva

Una **relazione ricorsiva** è una relazione binaria sulla stessa entità.

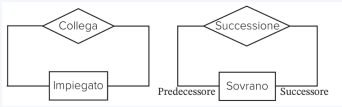


Figura 3.3: relazione ricorsiva

Definizione: relazione ternaria

Una **relazione ternaria** è una relazione che coinvolge tre entità e si usa quando:

- un concetto, per esistere, nel sistema informativo richiede sempre la presenza di tre istanze di entità (una per ogni entità coinvolta nella relazione)
- non esistono eccezioni al punto precedente
- non esistono situazioni nelle quali la terna di entità debba essere rappresentata più volte

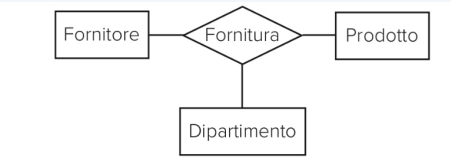
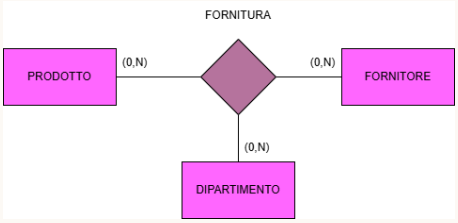


Figura 3.4: relazione tra tre entità

Un uso errato avviene quando per rappresentare lo stato del sistema informativo sarebbe necessario che un'istanza della relazione contenga solo una coppia e non una terna di istanze di entità (cioè quando manca un attore).
Quindi, in questo caso si può sostituire la relazione ternaria con tre relazioni binarie indipendenti.

Esempio: uso corretto della relazione ternaria

Vogliamo rappresentare il fatto che un profotto venga fornito ad un dipartimento da un fornitore.
Ogni fornitore può scegliere quali prodotti ordinare dai fornitori selezionando un sottoinsieme dei prodotti forniti dal fornitore.
Ogni fornitore vende a più dipartimenti prodotti diversi.
Ogni prodotto viene venduto da più fornitori.



dove le istanze hanno le seguenti caratteristiche:

- $I(\text{FORNITORE}) = \{1, 2, 3\}$
- $I(\text{PRODOTTO}) = \{p_1, p_2, p_3\}$
- $I(\text{DIPARTIMENTO}) = \{d_1, d_2\}$
- $I(\text{FORNITURA}) = \{(f_1, d_1, p_1), (f_1, d_2, p_3), (f_3, d_1, p_3), (f_3, d_2, p_2)\}$

3.3 Attributo

Definizione: attributo

Un **attributo** rappresenta una proprietà elementare di un'entità o di una relazione. Ogni attributo di un'entità o di una relazione associa ad ogni istanza un solo valore appartenente ad un dominio di valori ammissibili.

Può essere visto come una funzione che ha come dominio le istanze dell'entità (o relazione) e come codominio l'insieme dei valori ammissibili:

$$f_{\mathrm{A}} : \mathrm{I(E)} \rightarrow \mathrm{D}$$

dove:

A: attributo dell'entità E
I(E): insieme delle istanze di E
D: insieme dei valori ammissibili

La sintassi grafica di un attributo è un cerchio vuoto collegato con una linea all'entità con accanto il nome dell'attributo:

(a) attributo di un'entità **(b)** attributo di una relazione

Figura 3.5: attributo

Definizione: attributo opzionale e multivalore

L'**attributo opzionale** o **multivalore** si ottiene da un attributo normale specificando un vincolo di cardinalità sui valori che l'attributo può assumere (default: (1,1)).

I valori possibili sono i seguenti:

- ▷ **(0,1)**: attributo opzionale
- ▷ **(1,N)**: attributo multivalore obbligatorio
- ▷ **(0,N)**: attributo multivalore opzionale

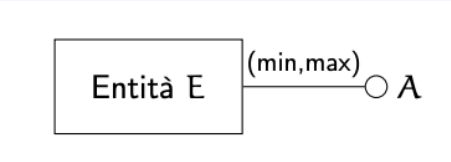


Figura 3.6: attributo opzionale-multivalore

Definizione: attributo composto

Un **attributo composto** permette di raggruppare gli attributi di un'entità o di una relazione che hanno un significato comune.

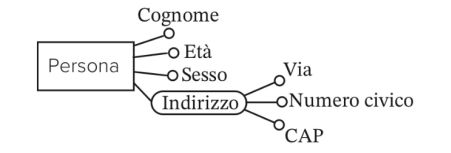


Figura 3.7: attributo composto

3.4 Identificatore

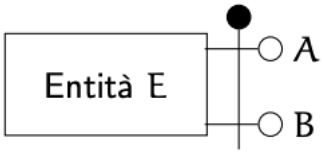
Definizione: identificatore

Data un'entità E, un **identificatore** è un insieme di proprietà (attributi e/o relazioni) che identificano univocamente ogni istanza di E.

Un insieme di proprietà identifica univocamente le istanze di un'entità E se non esistono due istanze di E che presentano gli stessi valori o istanze nelle proprietà dell'insieme.

La sintassi grafica di un identificatore per un'entità con un solo attributo è la seguente:

Per un identificatore costituito da più attributi a, b la sintassi grafica è la seguente:

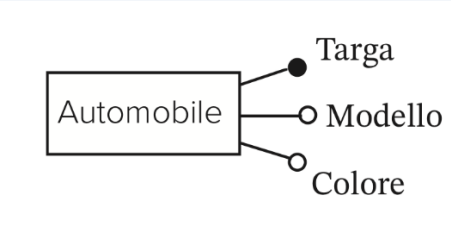


Può anche esistere un identificatore costituito da una relazione R (deve essere una funzione):

Oppure un identificatore può essere costituito soltanto da una relazione R:

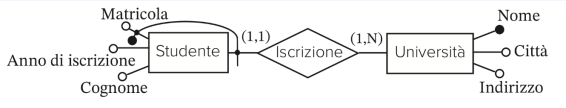
Definizione: identificatore interno

Un **identificate interno** è costituito solo da attributi dell'entità.



Definizione: identificatore esterno

Un **identificate esterno** è costituito da almeno una relazione con un'altra entità.

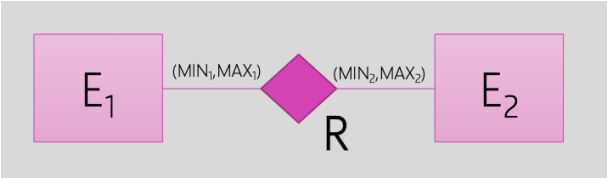


3.5 Cardinalità

Definizione: cardinalità

Data una relazione R i **vincoli di cardinalità** vengono specificati per ogni entità E_i coinvolta nella relazione R e specificano il numero minimo e massimo di istanze di R a cui un istanza di E_j deve o può partecipare.

La sintassi grafica per rappresentare i vincoli di cardinalità è la seguente:



Valori possibili per MIN_i

I possibili valori che il minimo MIN_i può assumere sono:

- ▷ **0**: partecipazione alla relazione R delle istanze di E_i è opzionale
- ▷ **1**: partecipazione alla relazione R delle istanze di E_i è obbligatoria
- ▷ **num** $>$ **1**: per ogni istanza di E_i devono essere presenti almeno num occorrenze

della relazione R che la coinvolgono

Valori possibili per $\mathrm{MAX_i}$

I possibili valori che il minimo $\mathrm{MAX_i}$ può assumere sono:

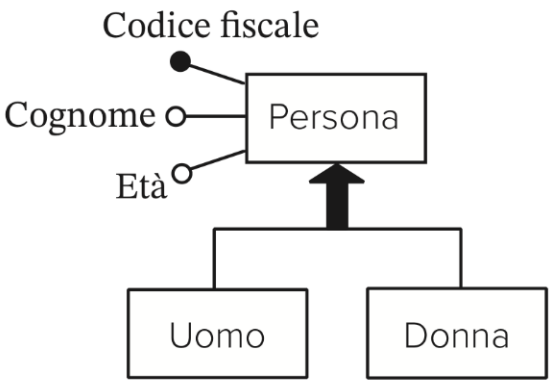
- **1**: un'istanza di $\mathrm{E_i}$ può al massimo partecipare a una sola occorrenza della relazione R (se R è binaria questo indica che R è una funzione)
- **N**: un'istanza di $\mathrm{E_i}$ può partecipare a più occorrenze della relazione R senza limite massimo
- **num** > 1: per ogni istanza di $\mathrm{E_i}$ possono essere presenti al massimo num occorrenze della relazione R che la coinvolgono

3.6 Generalizzazione di entità

Definizione: generalizzazione

La **generalizzazione** è un legame logico (simile ad un'ereditarietà tra classi) tra un'entità padre E e n ($n > 0$) entità figlie $(\mathrm{E_1}, \mathrm{E_2}, \ldots, \mathrm{E_n})$ dove E rappresenta una classe di oggetti più generale rispetto alle classi di oggetti rappresentate dalle entità figlie.

La sintassi grafica per rappresentare una generalizzazione è la seguente:



Proprietà delle istanze generalizzate

Le proprietà delle istanze generalizzate sono:

- ▷ ogni istanza di un'entità figlia $\mathrm{E_i}$ è anche istanza dell'entità padre E
- ▷ ogni proprietà (attributi, identificatori e relazioni) dell'entità padre E è anche proprietà di ogni entità figlia $\mathrm{E_i}$

Classificazione delle generalizzazioni

Una generalizzazione può essere classificata secondo due criteri:

- ◇ **totale**: se ogni istanza dell'entità padre E è anche istanza di almeno una delle entità figlie E_i, altrimenti si dice **parziale**
- ◇ **esclusiva**: se ogni istanza dell'entità padre E è anche istanza di al massimo una delle entità figlie E_i, altrimenti si dice **sovrapposta**

Per indicare la classificazione di una generalizzazione si mettono le seguenti notazioni accanto alla freccia della generalizzazione:

- ⋆ **(p,s)**
- ⋆ **(p,e)**: non si possono avere istanze comuni tra le entità figlie
- ⋆ **(t,s)**: non si possono avere istanze dell'entità padre che non siano istanze di almeno una delle entità figlie
- ⋆ **(t,e)**: ogni istanza dell'entità padre è istanza di esattamente una delle entità figlie

4

Progettazione logica

Nel corso degli anni sono stati sviluppati diversi modelli:

modello gerarchico/reticolare (anni '60-'70)
↓
modello relazionale (anni '80-oggi)
↓
modello ad oggetti/relazionale o **object-relstional** (anni 2000-oggi)
↓
modello a grago - RDF (anni 2005-oggi) ⇒ *Neo4j*
↓
modello noSQL (anni 2010-oggi) ⇒ *MongoDB*, ecc.

Il **modello relazionale** è un modello di dati che permette di definire le proprietà che la *popolazione* di una base di dati deve rispettare.

Proprietà del modello relazionale

Il modello relazionale è **value-based** (basato sui valori).

Il modello non prevede l'uso di puntatori per rappresentare legami tra i dati.
Questo implica:

- completa indipendenza dall'implementazione e dallo schema fisico
- facilità di trasferimento dei dati tra sistemi
- viene rappresentato solo ciò che è rilevante per il sistema informativo

I costrutti principali del modello relazionale sono:

- **domini di base**: insieme di valori atomici che rappresentano i tipi di dati elementari
- **relazione (o tabella)**: rappresenta un insieme di entità o di relazioni tra entità.
  Possono essere definite come:
  - insieme di **ennuple**
  - insieme di **tuple**

- **superchiavi**, **chiavi** e **chiavi primarie**: insieme di attributi che identificano univocamente ogni ennupla di una relazione
- **vincoli di integrità referenziale**: vincoli che garantiscono la coerenza tra i dati memorizzati in relazioni diverse
- **vincoli di integrità generici**: vincoli che garantiscono la coerenza dei dati memorizzati in una singola relazione

Definizione: domini di base

I **domini di base** sono i domini da cui si scelgono i valori delle proprietà delle istanze di informazione da rappresentare.

I domini tipici sono:
- *caratteri*
- *stringhe di caratteri*
- *numeri interi*
- *numeri deciamali a virgola fissa*
- *numeri decimali a virgola mobile*
- *domini del tempo*: rappresentano istanti e intervalli di tempo

Definizione: relazione

Una **relazione** può essere vista come una tabella.

Milano	20100	1.300.000
Verona	37100	350.000
Brescia	25100	250.000

Una tabella è un contenitore di dati la cui struttura è caratterizzata da una lista di colonne:
- ogni riga descrive le caratteristiche di un'istanza dell'informazione da rappresentare
- valori contenuti nelle colonne descrivono sempre la stessa proprietà delle istanze di informazione da rappresentare

Definizione: relazione (insieme di tuple - Lists)

Dati n insiemi di valori (*domini*) $D_1, \ldots, D_n$, con $n > 0$ e indicato con $D_1 \times \cdots \times D_n$ il loro prodotto cartesiano:

$$D_1 \times \cdots \times D_n = (v_1, \ldots, v_n) \mid v_1 \in D_1 \wedge \cdots \wedge v_n \in D_n$$

una relazione ρ di grado n è un qualsiasi sottoinsieme di $D_1 \times \cdots \times D_n$:

$$\rho \subseteq D_1 \times \cdots \times D_n$$

dove $(v_1, \ldots, v_n)$ è una ennupla della relazione e $|\rho|$ è la cardinalità della relazione (*numero di ennuple*).

N.B.

- ▷ domini $D_1, \ldots, D_n$ possono essere a cardinalità infinita, mentre le relazioni sono sempre a cardinalità finita
- ▷ non è definito alcun ordinamento sulle ennuple di una relazione
- ▷ non sono ammessi duplicati di una ennupla
- ▷ nella definizione di relazione come insieme di ennuple, i valori delle ennuple sono ordinati

Esempio: relazione come insieme di tuple (Lists)

Consideriamo la tabella:

Milano	20100	1.300.000
Verona	37100	350.000
Brescia	25100	250.000

La relazione può essere vista come insieme di ennuple (relazione città):

$$\rho = \left\{ \begin{array}{l} (\text{Milano}, 20100, 1300000) \\ (\text{Verona}, 37100, 350000) \\ (\text{Brescia}, 25100, 250000) \end{array} \right\}$$

dove:

$$\rho \subseteq D_1 \times D_2 \times D_3$$

Accesso ai valori di una tupla

Se t è una ennupla $(v_1, \ldots, v_n)$ il valore posto in i-esima posizione si indica con la notazione:

$$\mathbf{t[i]}$$

Questa modalità di accesso ai valori però non è efficace per l'uso pratico delle relazioni.
Si preferisce, quindi, assegnare un nome alle colonne.

Definizione: relazione (insime di tuple - Mappings)

Sia X un insieme di nomi e sia Δ l'insieme di tutti i domini di base ammessi dal modello.
Si definisce la funzione:

$$\mathbf{DOM : X \rightarrow \Delta}$$

Tale funzione associa ad ogni nome $A \in X$ un dominio $DOM(A) \in \Delta$.
I nomi di X si definiscono attributi.
Una tupla t su X è una funzione:

$$t : X \rightarrow \bigcup_{A \in X} DOM(A)$$

dove:

$$t[A] = v \in DOM(A)$$

N.B.

- una relazione è un insieme di tuple e quindi non può contenere tuple duplicate
- domini per gli attributi possono essere solo domini di base, non sono ammessi altri domini (non sono ammessi attributi annidati)
- una base di dati relazionale è composta da più relazioni

Esempio: relazione come insieme di tuple (Mappings)

Consideriamo la tabella:

Milano	20100	1.300.000
Verona	37100	350.000
Brescia	25100	250.000

Definiamo l'insieme di attributi:

$$X = \{nome, CAP, abitanti\}$$

Definiamo il dominio di ciascun attributo:

$$DOM(nome) = \text{stringhe di caratteri}$$
$$DOM(CAP) = \text{numeri interi}$$
$$DOM(abitanti) = \text{numeri interi}$$

La tabella della relazione è rappresentata come un insieme di tuple:

$$\rho_X = \{t_1, t_2, t_3\}$$

dove:

$$t_1[nome] = Milano \quad t_1[CAP] = 20100 \quad t_1[abitanti] = 1300000$$
$$t_2[nome] = Verona \quad t_2[CAP] = 37100 \quad t_2[abitanti] = 350000$$
$$t_3[nome] = Brescia \quad t_3[CAP] = 25100 \quad t_3[abitanti] = 250000$$

4.1 Progettazione nel modello relazionale

Nel modello relazione lo schema è costituito da un insieme di relazione (*tabelle*).

Le dipendenze tra proprietà elementari (*attributi*) sono costituite dalle dipendenze funzionali.

Requisiti da implementare nel modello relazionale

Si vuole rappresentare in una base di dati relazionale le informazioni sulla proprietà degli appartamenti siti nel comune di Verona.

Per ogni appartamento viene memorizzato: *codice ecografico* (univoco), *categoria catastale* e *indirizzo* (via, numero civico, ubalterno).

Per ogni proprietario si registra: *codice fiscale*, *nome*, *cognome* e *data di nascita*.

Un appartamento può essere in comproprietà e un proprietario può possedere più appartamenti.

4.1.1 Relazione unica

Una soluzione possibile è rappresentata da un'unica relazione che contiene tutte le informazioni sugli appartamenti e sui proprietari:

Categoria	Via	Civico	Sub	Codice ECO	Codice Fiscale	Nome	Cognome	Data Nas
A1	Roma	23	A	RMAXXX	RSSMRR75D	Mario	Rossi	1/1/1975
A1	Roma	23	A	RMAXXX	BNCMRA80F	Maria	Bianchi	10/9/1980
A3	Milano	9		MILYYY	BNCMRA80F	Maria	Bianchi	10/9/1980
A3	Milano	9		MILYYY	BNCPLO77A	Paolo	Bianchi	3/1/1977
A1	Torino	2	C	TORZZZ	BNCPLO77A	Paolo	Bianchi	3/1/1977
A2	Garibaldi	33		GARWWW	RSSMRR75D	Mario	Rossi	1/1/1975

Figura 4.1: schema relazionale con una sola relazione Proprietà

Questa tabella associa ogni appartamento ad un proprietario perchè i dati dell'appartamento e della persona fanno parte della stessa riga.

Problematiche

Il problema è che non c'è distinzione tra le due categorie di informazioni.

Questo porta a diversi problemi:

- **mancata separazione delle informazioni**: le due categorie di informazione non possono esistere separatamente
- **ridondanza dei dati**: se un proprietario possiede più appartamenti i suoi dati vengono memorizzati più volte.
  Questo comporta un aumento dello spazio di memorizzazione e una possibile inconsistenza dello stesso dato.

Le varie problematiche sono descritte in modo più dettagliato come **anomalie** e si dividono in:

- **anomalia di aggiornamento**: per aggiornare il valore di un attributo bisogna modificare tale valore su più tuple
- **anomalie di inserimento**: per inserire una nuova tupla è necessario inserire valori al momento sconosciuti (sostituibili da valori nulli) per gli attributi non disponibili
- **anomalie di cancellazione**: per cancellare una tupla è necessario cancellare valori ancola validi oppure inserire valori nulli per gli attributi da cancellare

4.1.2 Decomposizione della relazione unica

Consideriamo l'esempio precedente con una singola relazione e decomponiamola separando le diverse categorie di informazioni in relazioni distinte.

Metodo 1: separazione in due relazioni distinte

APPARTAMENTO

Categoria	Via	Civico	Sub	Codice ECO
A1	Roma	23	A	RMAXXX
A3	Milano	9		MILYYY
A1	Torino	2	C	TORZZZ
A2	Garibaldi	33		GARWWW

PROPRIETARIO

Codice Fiscale	Nome	Cognome	Data Nas
RSSMRR75D	Mario	Rossi	1/1/1975
BNCMRA80F	Maria	Bianchi	10/9/1980
BNCPLO77A	Paolo	Bianchi	3/1/1977

Questa separazione risolve i problemi di ridondanza e anomalie di aggiornamento. Manca, però, l'informazione che mette in relazione la persona con l'appartamento.

Questa decomposizione è detta **decomposizione con perdita**.

Metodo 2: separazione in due relazioni distinte + aggiunta di informazione sulla prima relazione

Si separano in due relazioni distinte *Appartamento* e *Proprietario* con l'aggiunta di un informazione identificante per la relazione *Appartamento* (codice fiscale della Persona).

PROPRIETARIO

Codice Fiscale	Nome	Cognome	Data Nas
RSSMRR75D	Mario	Rossi	1/1/1975
BNCMRA80F	Maria	Bianchi	10/9/1980
BNCPLO77A	Paolo	Bianchi	3/1/1977

APPARTAMENTO

Categoria	Via	Civico	Sub	Codice ECO	Codice Fiscale
A1	Roma	23	A	RMAXXX	RSSMRR75D
A1	Roma	23	A	RMAXXX	BNCMRA80F
A3	Milano	9		MILYYY	BNCMRA80F
A3	Milano	9		MILYYY	BNCPLO77A
A1	Torino	2	C	TORZZZ	BNCPLO77A
A2	Garibaldi	33		GARWWW	RSSMRR75D

Siccome non esistono i puntatori bisogna duplicare i valori identificanti nelle due relazioni.

Anche questa **decomposizione ha dati ridondanti**.

Metodo 3: separazione in due relazioni distinte + aggiunta di informazione sulla seconda relazione

Si separano in due relazioni distinte *Appartamento* e *Proprietario* con l'aggiunta di un'informazione identificante per la relazione *Proprietario* (codice ecografico dell'Appartamento).

PROPRIETARIO

Codice Fiscale	Nome	Cognome	Data Nas
RSSMRR75D	Mario	Rossi	1/1/1975
BNCMRA80F	Maria	Bianchi	10/9/1980
BNCPLO77A	Paolo	Bianchi	3/1/1977

APPARTAMENTO

Categoria	Via	Civico	Sub	Codice ECO	Codice Fiscale
A1	Roma	23	A	RMAXXX	RSSMRR75D
A1	Roma	23	A	RMAXXX	BNCMRA80F
A3	Milano	9		MILYYY	BNCMRA80F
A3	Milano	9		MILYYY	BNCPLO77A
A1	Torino	2	C	TORZZZ	BNCPLO77A
A2	Garibaldi	33		GARWWW	RSSMRR75D

Siccome non esistono i puntatori bisogna duplicare i valori identificanti nelle due relazioni. Anche questa **decomposizione ha dati ridondanti**.

Metodo 4: separazione in due relazioni distinte + aggiunta di informazione su entrmabe le relazioni

Si separano in due relazioni distinte *Appartamento* e *Proprietario* con l'aggiunta di un'informazione identificante per ciascuna relazione (codice fiscale della Persona e codice ecografico dell'Appartamento).

APPARTAMENTO

Categoria	Via	Civico	Sub	Codice ECO
A1	Roma	23	A	RMAXXX
A3	Milano	9		MILYYY
A1	Torino	2	C	TORZZZ
A2	Garibaldi	33		GARWWW

APPARTAMENTO

Categoria	Via	Civico	Sub	Codice ECO	Codice Fiscale
A1	Roma	23	A	RMAXXX	RSSMRR75D
A1	Roma	23	A	RMAXXX	BNCMRA80F
A3	Milano	9		MILYYY	BNCMRA80F
A3	Milano	9		MILYYY	BNCPLO77A
A1	Torino	2	C	TORZZZ	BNCPLO77A
A2	Garibaldi	33		GARWWW	RSSMRR75D

Siccome non esistono i puntatori bisogna duplicare i valori identificanti nelle due relazioni. Anche questa **decomposizione ha dati ridondanti**.

Metodo 5: separazione in tre relazione distinte

Si separano in tre relazione distinte:

1. *Appartamento*
2. *Persona*
3. *Proprietà* (relazione tra Appartamento e Persona)

Per rappresentare la relazione tra Appartamento e Persona si introduce una nuova relazione chiamata Proprietà che contiene i valori degli attributi identificanti di entrambe le relazioni.

APPARTAMENTO

Categoria	Via	Civico	Sub	Codice ECO
A1	Roma	23	A	RMAXXX
A3	Milano	9		MILYYY
A1	Torino	2	C	TORZZZ
A2	Garibaldi	33		GARWWW

PROPRIETARIO

Codice Fiscale	Nome	Cognome	Data Nas
RSSMRR75D	Mario	Rossi	1/1/1975
BNCMRA80F	Maria	Bianchi	10/9/1980
BNCPLO77A	Paolo	Bianchi	3/1/1977

Codice ECO	Codice Fiscale
RMAXXX	RSSMRR75D
RMAXXX	BNCMRA80F
MILYYY	BNCMRA80F
MILYYY	BNCPLO77A
TORZZZ	BNCPLO77A
GARWWW	RSSMRR75D

In questa decomposizione non esistono ridondanze e non si perdono informazioni.
Questa è la **decomposizione corretta e senza perdita**.
L'unica ridondanza è quella degli attributi identificanti che però è necessaria per rappresentare la relazione tra le due entità.

4.2 Valori nulli

Dalla definizione di relazione, si ha che ogni tupla deve sempre contenere nei propri attributi valori significativi appartenenti ai domini di base del modello.
Non sempre, però, in una base di dati reale esistono i valori per tutti gli attributi di una tupla.

Un valore può mancare nei seguenti casi:

- **valore di un attributo A inesistente** (attributo opzionale a livello concettuale): non esiste per questa tupla un valore per l'attributo A
- **valore di un attributo A è sconosciuto**: esiste un valore per l'attributo A di questa tupla ma non è noto alla base di dati
- **valore di un attributo A è inesistente o sconosciuto**

Per poter gestire tali situazioni viene introdotto nel modello relazionale un valore speciale detto **valore nullo**: gli attributi di una tupla possono assumere un valore del dominio oppure il valore nullo.

Definizione: tupla con valore nullo

Una tupla su X con valore nullo è definita come una funzione:

$$\mathbf{t : X \rightarrow \{NULL\} \cup (\bigcup_{A \in X} DOM(A))}$$

dove:

$$t[A] = v \in DOM(A) \vee t[A] = NULL$$

Osservazioni

La presenza di valori nulli è accettabile solo in alcuni attributi (non è possibile avere tuple di soli valori nulli).
In particolare, negli attributi replicati per rappresentare legami tra tuple la presenza di valori nulli può rendere inutilizzabile l'informazione.

Esempio: schema relazionale

Rappresentare attraverso un insieme di relazioni le informazioni contenute in un oriario ferroviario, dove si riportano per ogni treno in partenza dalla stazione di Verona Porta Nuova: *numero*, *orario di partenza*, *destinazione finale*, *categoria*, *fermate intermedie* (con orario di fermata) e *orario di arrivo* alla destinazione finale.
Lo schema relazionale è il seguente:

$$\text{TRENO}(\text{numero}, \text{categoria}, \text{oraPartenza}, \text{destinazione}, \text{oraArrivo})$$

$$\text{FERMATA}(\text{numTreno}, \text{stazione}, \text{orario})$$

In progettazione concettuale equivale a due entità (Treno e Stazione) collegate da una relazione (Fermata).

4.3 Vincoli d'integrità

Definizione: vincolo d'integrità

Un **vincolo di integrità** è una condizione, espressa da un predicato, che deve essere sempre soddisfatta da ogni istanza della base di dati.

4.3.1 Vincoli di dominio

Impongono una restrizione sui valori che un attributo può assumere.

Esempio: vincolo di dominio

- ▷ $\forall t \in \text{TRENO} : t[\text{oraPartenza}] \in \{0, 1, \ldots, 23\}$
- ▷ $\forall t \in \text{TRENO} : t[\text{minutoPartenza}] \in \{0, 1, \ldots, 59\}$
- ▷ $\forall t \in \text{TRENO} : t[\text{numero}] > 0$

4.3.2 Vincoli di tupla

Impongono una restrizione alla combinazione di valori che una tupla della relazione può assumere indipendentemente dalle altre tuple.

Esempio: vincolo di tupla

- $\forall t \in \text{TRENO} : t[\text{numero}] > 500 \Rightarrow t[\text{categoria}] = \text{'regionale'}$

4.3.3 Vincoli di intrarelazionali

Impongono una restrizione al contenuto di una relazione e specificano una condizione che ogni tupla della relazione deve soddisfare rispetto alle altre tuple della medesima relazione.

Una sottocateoria importante di questi vincoli include i **vincoli di chiave**.

Definizione: superchiave

Data una relazione di schema $R(X)$, un insieme di attributi K sottoinsieme di X è **superchiave** per $R(X)$ se per ogni istanza r di $R(X)$ vale la seguente condizione:

$$\forall \mathbf{t}, \mathbf{t'} \in \mathbf{r} : \mathbf{t} \neq \mathbf{t'} \Rightarrow \mathbf{t}[\mathbf{K}] \neq \mathbf{t'}[\mathbf{K}]$$

dove:

$$t[K] \neq t'[K] \equiv \exists A_i \in K : t[A_i] \neq t'[A_i]$$

Definizione: chiave candidata

Data una relazione di schema R(X), un insieme di attributi K sottoinsieme di X è chiave candidata (o chiave) per R(X) se K è superchiave per R(X) e vale la seguente condizione:

$$\neg \forall \mathbf{K'} \subseteq \mathbf{K} : \mathbf{K'} \text{ è superchiave per } R(X)$$

Definizione: chiave primaria

Data una relazione di schema $R(X)$ la sua chiave primaria è la chiave candidata scelta per identificare le tuple della relazione.
La chiave primaria viene usata per rappresentare i legami tra le relazioni.

Caratteristiche

- non contiene **mai** valori nulli
- su K il sistema genera una struttura d'accesso ai dati (o indice) per supportare le interrogazioni

La chiave primaria di una relazione $R(X)$ viene indicata sottolineando gli attributi della chiave.

Esempio: notazione della chiave primaria

La chiave primaria della tabella *Treno* è l'attributo numero:

TRENO(<u>numero</u>, categoria, oraPartenza, destinazione, oraArrivo)

La chiave primaria della tabella *Fermata* è:

FERMATA(<u>numTreno, stazione</u>, orario)

Esempio: vincoli di chiave

Consideriamo la tabella Treno.

I seguenti sottoinsiemi di X sono:

- $K1 = \{numero\}$: superchiave e chiave candidata
- $K2 = \{numero, categoria\}$: superchiave, ma non chiave candidata
- $K3 = \{oraPartenza, minutoPartenzart, destinazione, categoria\}$: superchiave e chiave candidata

4.3.4 Vincoli interrazionali

Impongono una restrizione al contenuto di una relazione e specificano una condizione che ogni tupla della relazione deve soddisfare rispetto alle tuple di altre relazioni della base di dati.

Una sottocategoria importante di tali vincoli include i **vincoli d'integrità referenziale** (o vincoli sulle chiavi esportate).

Definizione: vincoli d'integrità referenziale

Un vincolo di integrità referenziale tra un insieme di attributi $Y = \{A_1, ..., A_p\}$ di R_1 e un insieme di attributi $K = \{K_1, ..., K_p\}$, chiave primaria di un'altra relazione R_2, è soddisfatto se, per ogni istanza r_1 di R_1 e per ogni istanza r_2 di R_2 vale la seguente condizione:

$$\forall t \in r_1 : \exists s \in r_2 : \forall i \in \{1, \ldots, p\} : t[A_i] = s[K_i]$$

In presenza di valori nulli (legame opzionale) la condizione diventa:

$$\begin{aligned} \forall t \in r_1 : \ \exists s \in r_2 : & \\ & \Big((\forall i \in \{1, \ldots, p\} : t[A_i] = s[K_i]) \\ & \vee (\exists i \in \{1, \ldots, p\} : t[A_i] = NULL) \Big) \end{aligned}$$

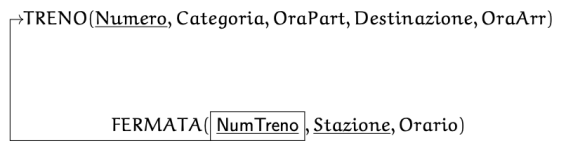
Il vincolo di integrità referenziale viene indicato in due modi.

Metodo 1

Riquadrando gli attributi soggetti al vincolo e collegando con una freccia il riquadro alla relazione (tabella) da cui la chiave proviene.

Esempio

Supponendo che esista un vincolo di integrità referenziale sull'attributo numTreno della tabella *Fermata* rispetto alla tabella *Treno*, la notazione per indicarlo è la seguente:



Metodo 2

Indicare con una freccia l'attributo soggetto al vincolo verso la relazione da cui la chiave proviene, ad esempio:

$$R_1, D \rightarrow R_2$$
$$R_3(Y, X) \rightarrow R_1$$

L'attributo opzionale è indicato con un asterisco accanto al nome dell'attributo.

4.4 Progettazione logica dallo schema concettuale

L'obiettivo della progettazione logica è quello di produrre uno schema logico che descriva in modo corretto ed efficace tutte le informazioni contenute nello schema concettuale.

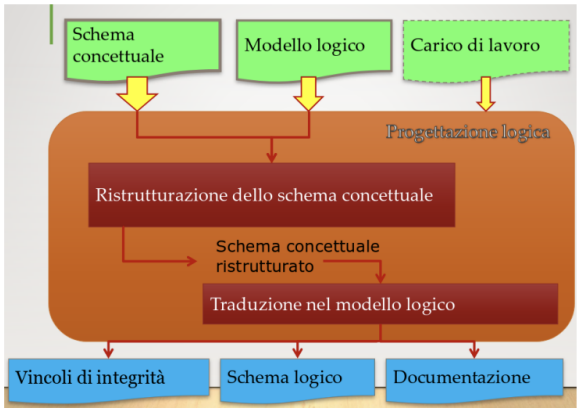


Figura 4.6: diagramma della progettazione logica

4.4.1 Ristrutturazione dello schema logico

La ristrutturazione dello schema concettuale può essere necessaria per i seguenti motivi:

- **semplificare la fase successiva di traduzione**
- **ottimizzare le strutture rispetto al carico di lavoro**:
  - dimensioni dei dati
  - caratteristiche delle operazioni:
    - costo (numero di accessi alla memoria secondaria)
    - frequenza

4.4.1.1 Analisi delle ridondanze per la presenza di dati derivabili

L'obiettivo dell'analisi delle ridondanze è quello di analizzare i dati derivabili presenti nello schema e decidere se:

- ⋆ mantenere il dato derivabile (memorizzandolo esplicitamente)
- ⋆ calcolare il dato derivabile quando serve

Esempio: tipologie di dati derivabili

- ▷ attributi derivabili da altri attributi della stessa entità (o relazione)
- ▷ attributi derivabili da attributi di altre entità (o relazioni)
- ▷ attributi derivabili dal conteggio di occorrenze

4.4.1.2 Eliminazione delle generalizzazioni

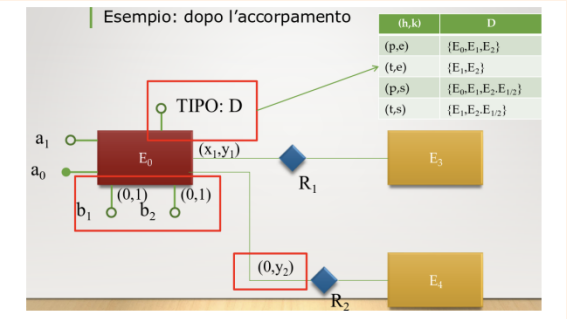
L'obiettivo è quello di sostituire le generalizzazioni presenti nello schema E-R con strutture alternative, in quanto non direttamente rappresentabili nel modello relazionale.

Esistono tre possibili soluzioni.

Metodo 1: accorpamento delle entità figlie nel padre

I passi sono i seguenti:

1. eliminazione delle entità figlie
2. accorpamento nel padre di tutti gli attributi specifici delle entità figlie come attributi opzionali
3. accorpamento nel padre delle relazioni che coinvolgono le entità figlie assegnando cardinalità minima uguale a zero (lato entità padre)
4. aggiunta di un attributo "tipo" per distinguere nel padre le occorrenze delle entità figlie eliminate



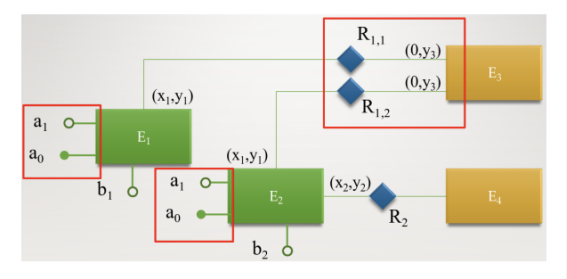
Metodo 2: accorpamento del padre nelle entità figlie

I passi sono i seguenti:

1. eliminazione dell'entità padre
2. replicazione degli attributi dell'entità padre su ogni entità figlia
3. partizionamento sui figli delle relazioni che coinvolgono l'entità padre assegnando

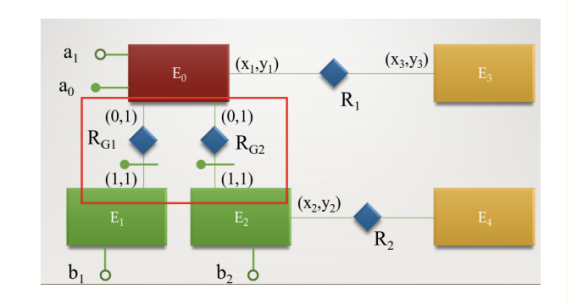
cardinalità minima uguale a zero (lato entità esterne)

N.B.
Per poter applicare questa trasformazione la generalizzazione deve essere totale.



Sostituzione della generalizzazione con relazioni

I passi sono i seguenti:
1. eliminazione della generalizzazione
2. inserire n relazioni tra il padre e ciascuna delle n entità figlie
3. tutti gli attributi conservano la loro collocazione



4.4.1.3 Accorpamento (o partizionamento) di entità e relazioni

Il **partizionamento di entità** è la sostituzione di un'entità dello schema con una coppia di entità con lo stesso identificatore e una relazione uno a uno che le lega tra loro.

Si esegue solo nel caso in cui esistano operazioni frequenti che trattano solo un sottoinsieme degli attributi dell'entità da partizionare.

Il **partizionamento di relazioni** si realizza sostituendo una relazione R con due relazioni distinte R_1 e R_2, dove le occorrenze di R si partizionano tra R_1 e R_2.
È conveniente quando esistono operazioni che si riferiscono ad un sottoinsieme delle occorrenze di R e altre operazioni che si riferiscono alle altre occorrenze di R.

4.4.1.4 Scelta degli identificatori principali

Va scelto un identificatore principale per ogni entità in quanto tale identificatore verrà utilizzato nel modello relazionale per identificare le tuple che rappresentano istanze di entità.

Si scelgono seguendo i seguenti criteri:

- ▷ gli identificatori che includono attributi opzionali non possono essere chiavi primarie
- ▷ meglio identificatori con pochi attributi
- ▷ meglio identificatori utilizzati nelle operazioni

4.4.2 Traduzione nel modello logico

La traduzione è un processo automatico che si realizza applicando allo schema concettuale ristrutturato un insieme di regole di traduzione.
Ogni regola si applica ad un costrutto deo modello concettuale ER e produce una o più strutture del modello relazionale:

- si rappresenta un'istanza di entità con una tupla
- si rappresenta un'istanza di relazione con un legame tra tuple o con una tupla esplicita
- si rappresenta l'identificatore principale di un'entità con la chiave primaria della tabella che implementa l'entità

Le regole di traduzione sono le seguenti:

5

Tecnologie NoSQL

I vari sistemi NoSQL sono molto diversi tra loro in termini di rappresentazione dei dati e di linguaggi per la loro manipolazione.
Nonostante questa diversificazione delle soluzioni attualmente disponibili, è possibile classificare la maggior parte dei sistemi NoSQL in alcune categorie principali:

sistemi chiave-valore: ogni dato è costituito da una coppia $(chiave, valore)$.

chiave: identifica le istanze

valore: qualsiasi struttura (anche non omogenea)

La principale modalità di accesso offerta è sulla base di un valore per la chiave.
Il campo *valore* associato a una *chiave* è tipicamente opaco per l'utente e non esistono meccanismi di ricerca basati sul suo contenuto.
Non esiste, quindi, uno schema a cui riferirsi e le primitive di accesso ai dati sono in genere molto semplici.

⇒ **Hadoop**

sistemi basati su documenti (document store): dati rappresentati sotto forma di documento strutturato e nidificato, tipicamente utilizzando il formato standard JSON.
Ogni oggetto della collezione ha una struttura complessa senza schema fisso.
Consentono la definizione di indici secondari su proprietà degli oggetti.
Le primitive di accesso ai dati consentono di interrogare una base di dati sulla base dei campi e sottocampi di questi documenti.

⇒ **MongoDB**, **CouchBase**

sistemi basati su colonne (column store): dati sono raggruppati in record (*righe*), fatti da una serie di campi (*colonne*) la cui quantità e tipologia può variare da record a record.
L'accesso ai dati avviene specificando le colonne di interesse (proiezione) e/o valori per queste colonne (selezione).
La rappresentazione è quindi affine al modello relazionale (righe e colonne), ma senza alcun vincolo sulla struttura delle righe.

⇒ **BigTable** di Google, **HBase**, **Cassandra**

sistemi a grafo (graph store): i dati vengono rappresentati in forma di grafo.

nodi: rappresentano gli oggetti

archi: rappresentano le relazioni tra questi o proprietà

Le operazioni di accesso ai dati si basano su primitive di navigazione nel grafo a partire da uno o più nodi iniziali.
Si tratta di sistemi particolarmente adatti quando si vuole gestire una base di dati che ha una naturale rappresentazione mediante un grafo.
⇒ **Neo4j**, **OrientDB**

5.1 Sistemi basati su documenti (document store)

5.1.1 Caratteristiche principali

Nei sistemi basati su documenti, gli oggetti memorizzati in una base di dati possono avere anche una **struttura complessa** (no tuple piatte) e possono essere acceduti facendo rierimento a componenti interne dell'oggetto, senza necessariamente coinvolgere la chiave primaria.

I meccanismi per gestire e manipolare i dati offerti sono più flessibili, pur garantendo elevate qualità di efficienza e scalabilità su infrastutture basate su cluster di computer.

I dati sono organizzati in collezioni di **informazioni omogenee** per le quali però non esite un rigido schema di riferimento.
Ogni **collezioni** contiene **oggetti** (*documenti*) che sono rappresentati utilizzando tipicamente un formato "auto-descritttivo" standard (Json), possono avere una struttura complessa di tipo gerarchico e sono concettualemnete **simili**, **ma non identici tra loro**.

5.1.2 Progettazione dei dati come documenti

La decisione importante nel processo di progettazione dei dati usando questo nuovo approcio riguarda:

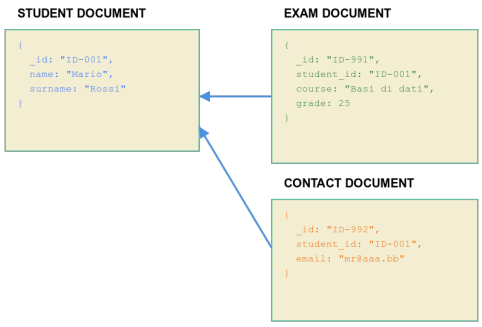
- scelta della chiave di accesso
- struttura dei documenti (*grado di incapsulamento*)
- rappresentazione delle relazioni (*legami*) tra documenti

Esistono due approci per rappresentare le relazioni tra documenti:

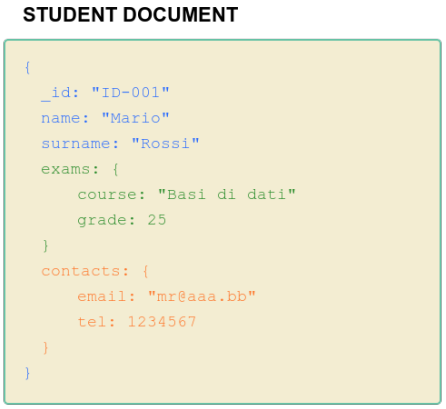
1. **tramite riferimenti**: si ottengono esportando l'ID del documento
2. **tramite documenti nidificati** (*incapsulamento*)

L'**uso di riferimenti** per rappresentare i legami tra documenti porta a schemi normalizzati.

L'**uso di documenti nidificati** porta a schemi non normalizzati e parzialmente rindondanti, ma molto efficienti in lettura.



(a) Legami mediante riferimenti



(b) Legami mediante documenti nidificati

L'**approcio normalizzato** consente di rappresentare il dato in modo simile a quanto avviene nel modello relazionale: non c'è rindondanza e gli aggiornamenti sono più semplici.

L'**approcio embedded** consente di rappresen in un unico documento quanto viene rappresentato di solito nel modello E-R con un'entità E insieme alle sue "entità deboli" (entità collegate ad E attraverso identificatori esterni).
E' posssibile rappresentare nello stesso documento un'istanza di informazione con altre istanze in essa logicamente contenute.

5.1.3 Traduzione da schema E-R a collezione di documenti

Supponiamo di applicare un **approcio embedded** a uno schema E-R già ristrutturato (privo di generalizzazioni).
Le relazioni ternarie (o ennarie) devono essere eliminate.
A partire dalle operazioni di elaborazione da eseguire sui dati vengono identificate le **collezioni di documenti da generare nel sistema**:

$$\{\mathrm{DOC}_1, \ldots, \mathrm{DOC}_\mathrm{N}\}$$

Possono essere al massimo una per ogni entità dello schema.

Regole di base

- ogni collezione DOC_i è in corrispondenza con un'entità EP dello schema E-R, dettà entità principale della collezione di documenti
- ogni istanza delle entità non principali sarà rappresentato da uno o più documenti incapsulato in un altro documento che rapppresenta un'istanza di un'entità principale
- le istanze di relazione saranno rapppresentate strutturalmente oppure esportando ID di documento o chiavi primarie

5.1.3.1 Prima fase: individuazione delle entità principali

Consiste nell'etichettare l'entità principale di una collezione di documenti DOC_i con l'etichetta DOC_i.

Per esempio, se la collezione DOC_3 ha come entità principale l'entità E_2, questa va etichettata nel seguente modo:

Figura 5.2: entità con proprio documento

5.1.3.2 Seconda fase: incapsulamento delle entità non principali

Per ogni entità E_i non etichettata come principale, va deciso in quale collezione incapsularla.

Casi possibili

(2.1) $\mathbf{E_i}$ è collegata attraverso una relazionale **R** ad una sola entità principale **EP**:

$\mathbf{E_i}$ **verrà incapsulato in EP**

(2.2) $\mathbf{E_i}$ è collegata attraverso le relazioni $\mathbf{R_1, \ldots, R_N}$ a più entità principali $\mathbf{EP_1, \ldots, EP_N}$.
Occorre in questo caso scegliere in quale entità principale EP_x incapsulare E_i:

$\mathbf{E_i}$ **verrà incapsulata in** $\mathbf{EP_X}$

(2.3) $\mathbf{E_i}$ non è collegata attraverso una relazione **R** ad un'entità principale, ma è collegata ad altre entità $\mathbf{E_{y1}, \ldots, E_{yN}}$ già collegate ad altre entità principali.
Occore in questo caso scegliere l'entità $\mathrm{E_{yK}}$ in cui incapsulare $\mathrm{E_i}$:

$\mathbf{E_i}$ **verrà incapsulato in** $\mathbf{E_{yK}}$

Per indicare che l'entità E_i viene incapsulata in **EP** si usa la freccia $\Downarrow$.

Casi possibili nelle situazioni (2.1) e (2.2)

E_i è collegata attraverso una relazione **R** as un'entità principale **EP** (unica o scelta tra quelle possibili):

$\mathbf{E_i}$ **verrà incapsulata in EP**

$\Rightarrow \mathrm{E_i}$ partecipa alla relazione R con vincolodi cardinalità (1,1)

Figura 5.3: X: incapsulamento senza perdita e rindondanze

⇒ E_i partecipa alla relazione R con vincolo di cardinalità (0,1)

Figura 5.4: XL (L = loss): incapsulamento con perdita

⇒ E_i partecipa alla relazione R con vincolo di cardinalità (1,N)

Figura 5.5: XR: incapsulamento con rindondanza

⇒ E_i partecipa alla relazione R con vincolo di cardinalità (0,N)

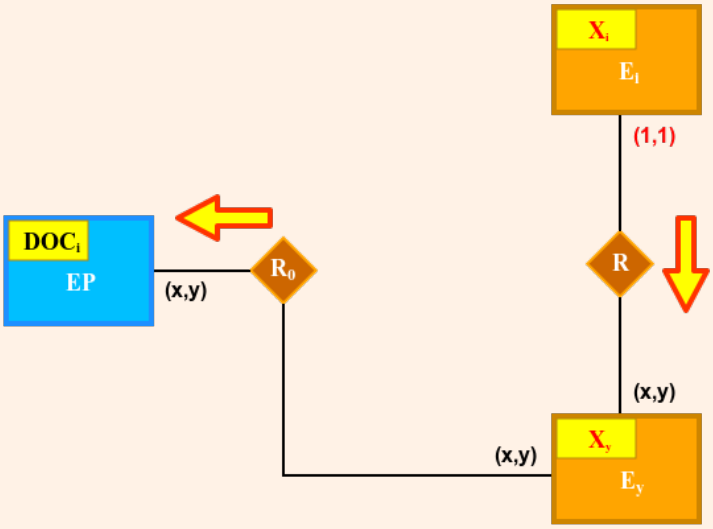
Figura 5.6: XLR: incapsulamento con perdita e rindondanza

Casi possibili nella situazione (2.3)

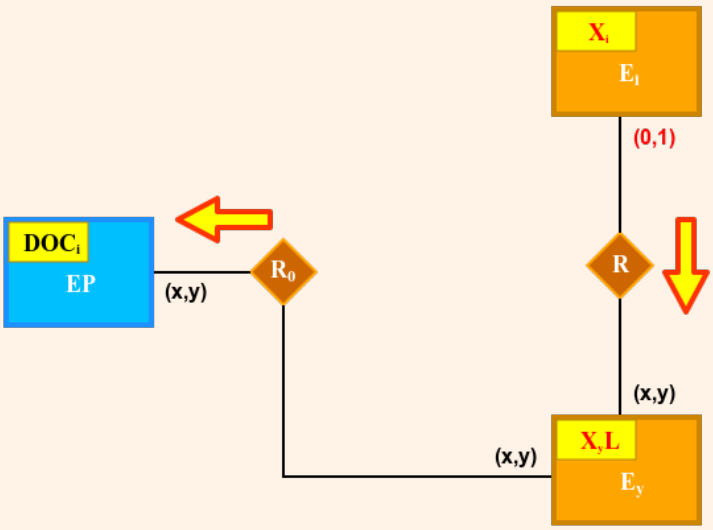
$\mathbf{E_i}$ è collegata attraverso una relazione **R** ad un'entità $\mathbf{E_y}$ (unica o a scelta) a sua volta collegata con un'entità **EP**:

$\mathbf{E_i}$ **verà incapsulata** $\mathbf{E_y}$

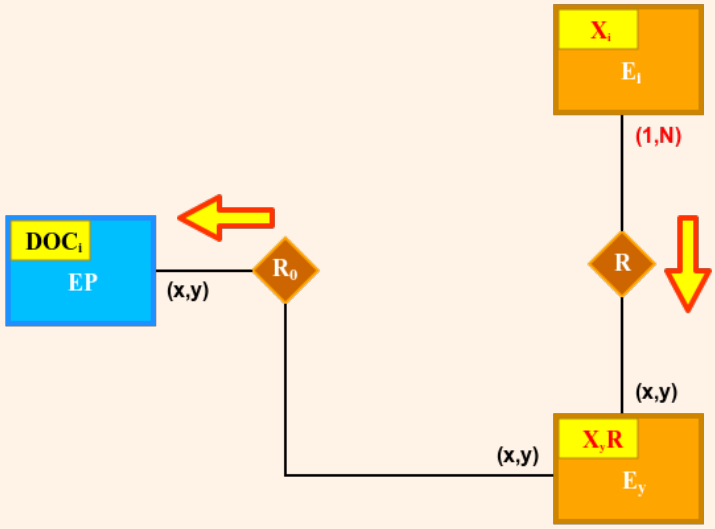
⇒ E_i partecipa alla relazione R con vincolodi cardinalità (1,1)



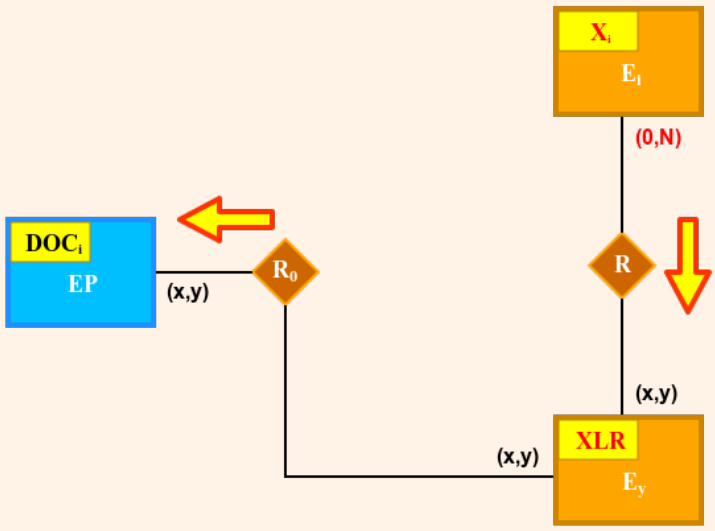
⇒ E_i partecipa alla relazione R con vincolodi cardinalità (0,1)



⇒ E_i partecipa alla relazione R con vincolodi cardinalità (1,N)



$\Rightarrow$ E_i partecipa alla relazione R con vincolodi cardinalità (0,N)

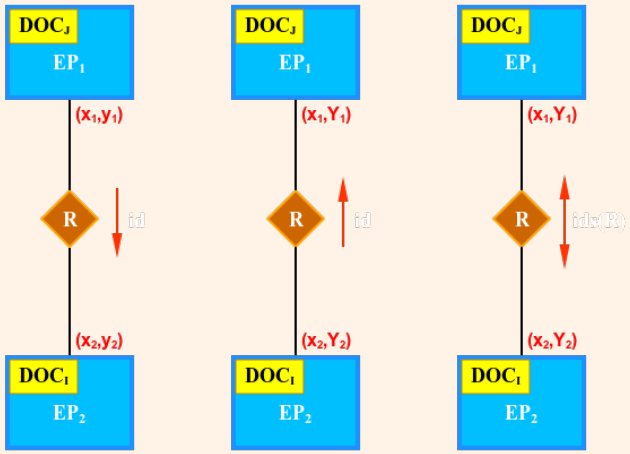


5.1.3.3 Terza fase: rappresentazione delle relazioni non già rappresentate strutturalmente

Tutte le relazioni che sono state etichettate da una freccia piena sono già rappresentate strutturalmente.
Quelle che rimangono devono essere rapppresentate con il meccanismo dei riferimenti.

Casi standard

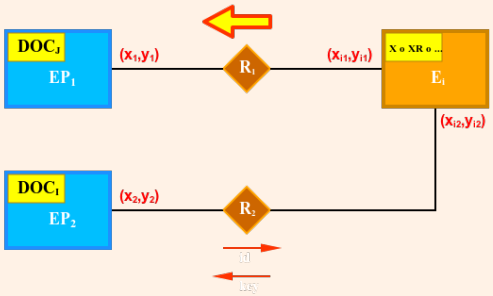
Per ogni relazione binaria **R** che coinvolge due entità principali EP_1 e EP_2, va deciso in quale collezione incapsularla.
I casi possibili sono:
⇒ **R** viene incapsulata in una sola entità principale $\mathbf{EP_1}$ (o $\mathbf{EP_2}$)
⇒ **R** viene incapsulata in entrambe le entità principali $\mathbf{EP_1}$ e $\mathbf{EP_2}$ (genera rindondanza)



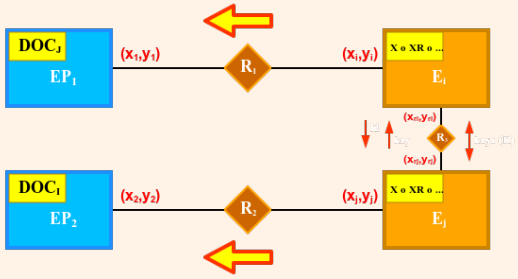
Viene utilizzata una freccia semplice per indicare dove viene rappresentata la relazione e la sua etichetta indica se la rappresentazione è per l'id del documento (ID) o per la chiave applicativa (KEY).
Un suffisso "(R)" indica che la rappresnetazione è rindondnate.

Casi particolari di seconda e/o terza fase

⇒ un'entità non principale $\mathbf{E_i}$ (incapsulata in un'entità principale EP_1) è collegata attraverso una relazione binaria **R** ad un'altra entità principale $\mathbf{EP_2}$



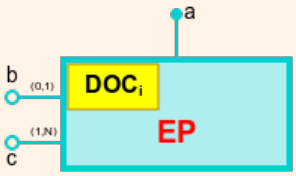
⇒ un'entità non principale $\mathbf{E_i}$ (incapsulata in un'entità principale EP_1) è collegata attraverso una relazione binaria **R** ad un'altra entità non principale $\mathbf{E_y}$ (incapsulata in un'altra entità principale EP_2)



5.1.3.4 Quarta fase: regole di traduzione di uno schema etichettato

Le strutture dei documenti vengono descritte con una sintassi semplificata e la generazione della struttura dipende poi dallo specifico sistema scelto.

Regola 1: entità principale




```
DOC_I
{
  _id: autogen,
  a : string,
  b*: integer,
  c : [number]
}
```

Gli attributi sono definiti come:

$$\langle \text{nome_attributo} \rangle : \langle \text{TIPO} \rangle$$

dove:

⟨TIPO⟩: può essere **string** | **integer** | **number** | **boolean** | **date** | **time** | **timestamp**

_id: sempre presente e generato automaticamente dal sistema

Regola 2: entità non principale

⇒ un'entità principale **EP** con un'entità debole $\mathbf{E_i}$ (o non principale) da incapsulare, allora E_i partecipa alla relazione **R** con vincolo di cardinalità (1,1) e **EP** con vincolo di cardinalità (X,N).

⇒ un'entità principale **EP** con entità $\mathbf{E_i}$ (o non principale) da incapsulare, allora E_i partecipa alla relazione **R** con vincolo di cardinalità (1,1) r **EP** con vincolo di cardinalità (X,1)

Regola 3: entità debole o non principale

⇒ un'entità principale **EP** con un'entità debole $\mathbf{E_i}$ (o non principale) da incapsulare, allora $\mathrm{E_i}$ partecipa alla relazione **R** vincolo di cardinalità (0,1) o (0,N) o (1,N) e **EP** con vincolo di cardinalità (X,N)
⇒ un'entità principale **EP** con un'entità debole $\mathbf{E_i}$ (o non principale) da incapsulare, allora $\mathrm{E_i}$ partecipa alla relazione **R** vincolo di cardinalità (0,1) o (0,N) o (1,N) e **EP** con vincolo di cardinalità (X,1)

Regola 4: entità principale incapsulata in un'entità non principale

⇒ $\mathbf{E_i}$ è collegata attraverso una relazione **R** ad un'entità $\mathbf{E_y}$ (unica o a scelta) che a sua volta collegata con un'entità **EP**, allora Ei viene incapsulata in $\mathrm{E_y}$ e $\mathrm{E_y}$ viene incapsulata in **EP**.
I vincoli su E_y e **EP** sono (0,N) o (1,N):
⇒ $\mathbf{E_i}$ è collegata attraverso una relazione **R** ad un'entità $\mathbf{E_y}$ (unica o a scelta) che a sua volta collegata con un'entità **EP**, allora Ei viene incapsulata in $\mathrm{E_y}$ e $\mathrm{E_y}$ viene incapsulata in **EP**.
I vincoli su $\mathrm{E_y}$ e **EP** sono (0,1) o (1,1):

Regola 5: relazione molti a molti tra entità principali

Se c'è una relazione molti a molti tra entità principali $\mathbf{EP_1}$ ed $\mathbf{EP_2}$, allora $\mathrm{EP_2}$ partecipa alla relazione R con vincolo di cardinalità (0,N) o (1,N) e incapsulo **R** solo in $\mathrm{EP_2}$:

Regola 6: relazione binaria tra entità non principale ed entità principale

⇒ relazione binaria $\mathbf{P_2}$ tra un'entità non principale $\mathbf{E_i}$, da incapsulare in $\mathbf{EP_1}$, e un'entità principale $\mathbf{EP_2}$ con vincolo (X,1) lato $\mathrm{E_i}$
⇒ relazione binaria $\mathbf{P_2}$ tra un'entità non principale $\mathbf{E_i}$, da incapsulare in $\mathbf{EP_1}$, e un'entità principale EP2 con vincolo (X,N) lato $\mathrm{E_i}$

Regola 7

⇒ relazione binaria $\mathbf{P_2}$ tra due entità non principali $\mathbf{E_i}$, $\mathbf{E_j}$, da incapsulare in $\mathbf{EP_1}$ ed $\mathbf{EP_2}$ rispettivamente, con vincolo (X,1) lato $\mathrm{E_i}$:
⇒ relazione binaria $\mathbf{P_2}$ tra due entità non principali $\mathbf{E_i}$, $\mathbf{E_j}$, da incapsulare in $\mathbf{EP_1}$ ed $\mathbf{EP_2}$ rispettivamente, con vincolo (X,N) lato $\mathrm{E_i}$:

6

Operazioni su una base di dati relazionale

I linguaggi per gestire i dati sono divisi in due categorie a seconda del tipo di operazioni che consentono di eseguire:

- **Data Definition Language (DDL)**: usato per la **definizione** dei dati.
  Questo linguaggio consente di:
  - creare lo schema della base di dati
  - modificare lo schema della base di dati
- **Data Manipulation Language (DML)**: usato per la **manipolazione** dei dati.
  Questo linguaggio consente di:
  - inserire o cancellare tuple nelle relazioni
  - aggiornare i valori nelle tuple delle relazioni
  - interrogare le relazioni per estrarre informazioni

I linguaggi di interrogazione che fanno parte del DML si dividono in due categorie:

- **linguaggi procedurali**: specificano il procedimento per ottenere il risultato (es. algebra relazionale)
- **linguaggi dichiarativi**: specificano le proprietà del risultato (es. calcolo relazionale e SQL)

6.1 Algebra relazionale

L'**algebra relazionale** è un linguaggio procedurale per l'interrogazione delle basi di dati relazionali.
Questo linguaggio è un insieme di operatori su relazioni.

"Segnatura" degli operatori

- **operatore unario**

$$\mathbf{op_{parametri}(r_1) \rightarrow r_2}$$

dove:
- ⋆ r_1 e r_2: relazioni
- ⋆ op: operatore con parametri specifici

▷ **operatore binario**

$$\mathbf{r_1 op_{parametri}(r_2) \rightarrow r_3}$$

dove:
- ⋆ r_1, r_2 e r_3: relazioni
- ⋆ op: operatore con parametri specifici

Gli operatori si possono classificare in due modi:

◇ **operatori funzionali**:
- operatori insiemistici
- operatori specifici
- operatori di giunzione (o join) tra relazioni

◇ **operatori derivabili**:
- operatori di base
- operatori derivati

6.1.1 Operatori insiemistici

Siccome le relazioni sono insiemi di tuple ha senso applicare ad esse le operazioni insiemistiche. Tuttavia va considerato che le relazioni sono insiemi di tuple omogenee, cioè tutte le tuple di una relazione hanno lo stesso schema (stesso insieme di attributi con stessi domini).
Quindi le operazioni insiemistiche si possono applicare soltanto a relazioni con lo stesso schema.

Date due relazioni r_1 e r_2 sullo stesso schema X si definiscono i seguenti operatori.

Definizione: operatori di base

▷ **unione**:

$$r_1 \cup r_2 = r_3 = \{t \mid t \in t \vee t \in r_2\}$$

▷ **differenza**:

$$r_1 - r_2 = r_3 = \{t \mid t \in r_1 \wedge r_1 t \notin r_2\}$$

N.B.
→ unione: tutte le tuple duplicate vengono rappresentate una sola volta
→ differenza: non è commutativa

Definizione: operatori derivati

▷ **intersezione**:

$$r_1 \cap r_2 = r_1 - (r_1 - r_2)$$

La cardinalità degli operatori insiemistici è il numero di tuple contenute nella relazione risultante:

- **unione**:
$$\max(|\mathrm{r_1}|, |\mathrm{r_2}|) \leq |\mathrm{r_1} \cup \mathrm{r_2}| \leq |\mathrm{r_1}| + |\mathrm{r_2}|$$

- **differenza**:
$$0 \leq |\mathrm{r_1} - \mathrm{r_2}| \leq |r_1|$$

- **intersezione**:
$$0 \leq |\mathrm{r_1} \cap \mathrm{r_2}| \leq \min(|\mathrm{r_1}|, |\mathrm{r_2}|)$$

6.1.2 Operatori specificici

Data una relazione $\mathrm{r_1}$ sullo schema $\mathrm{X} = \{\mathrm{A_1}, \mathrm{A_2}, \ldots, \mathrm{A_n}\}$, si definiscono i seguenti operatori (tutti unari).

Definizione: operatori di base

▷ **ridenominazione**: cambia il nome degli attributi di una relazione.
Sia $\mathrm{Y} = \{\mathrm{B_1}, \mathrm{B_2}, \ldots, \mathrm{B_n}\}$ un insieme di attributi con $|\mathrm{Y}| = |\mathrm{X}|$, allora:

$$\rho_{\mathrm{A_1},\ldots,\mathrm{A_n} \to \mathrm{B_1},\ldots,\mathrm{B_n}}(r_1) = r_2 = \{\mathrm{t} \mid \exists \mathrm{t}' \in \mathrm{r_1} : \forall i \in \{1, \ldots, \mathrm{n}\},\ \mathrm{t}[\mathrm{B_i}] = t'[\mathrm{A_i}]\}$$

▷ **selezione**:consente di estrarre da una relazione le tuple che soddisfano una certa condizione $\mathrm{F(t)}$:
$$\sigma_{\mathrm{F}}(\mathrm{r_1}) = (r_2) = \{\mathrm{t} \mid \mathrm{t} \in \mathrm{r_1} \wedge F(t)\}$$

La condizione F(t) è una formula proposizionale con le seguenti caratteristiche:

◇ si ottiene combinando con i connettivi logici $\wedge$, $\vee$ e $\neg$ condizioni atomiche del tipo $A\ \Theta\ B$ o $A\ \Theta\ c$, dove:
- $\mathrm{A}, \mathrm{B} \in \mathrm{X}$
- $\Theta \in \{<, =, >, \neq, \geq, \leq\}$
- $\mathrm{c} \in \mathrm{DOM(A)}$ è una costante compatibile con il dominio di A

◇ una formula $\mathrm{A}\ \Theta\ \mathrm{B}$ è vera sulla tupla t se:
$$\mathrm{t[A]}\ \Theta\ \mathrm{t[B]}$$

◇ una formula $\mathrm{A}\ \Theta\ \mathrm{c}$ è vera sulla tupla t se:

◇
$$\mathrm{t[A]}\ \Theta\ \mathrm{c}$$

◇ le formule $\mathrm{F_1} \wedge \mathrm{F_2}$, $\mathrm{F_1} \vee \mathrm{F_2}$ e $\neg\mathrm{F}$ hanno valori booleani definiti dalle note tabelle di verità dei connettivi logici corrispondenti

▷ **proiezione**: riduce le tuple di una relazione in verticale, cioè elimina attributi. Sia $Y = \{A_1, \ldots, A_m\} \subseteq X$ un sottoinsieme degli attributi di X allora:

$$\Pi_Y(r_1) = r_2 = \{t \mid \exists t' \in r_1 \ : \ t'[Y] = t\}$$

dove $t'[Y]$ indica una tupla t_0 su Y tale che:

$$\forall A_i \in Y \ : \ t_0[A_i] = t'[A_i]$$

Esempio: opertore di ridenominazione

Consideriamo lo schema relazionale:

$$\text{CAPOLUOGO_PROV}(\text{nome}, \text{popolazione}, \text{CAP})$$

$$\text{COMUNI_MINORI}(\text{nome}, \text{abitanti}, \text{CAP})$$

Si richiede di produrre la lista di tutti i comuni:

$$\boldsymbol{\rho_{\text{popolazioni}\rightarrow\text{abitanti}}(\text{CAPOLUOGO_PROV}) \cup \text{COMUNI_MINORI}}$$

Esempio: operatore di selezione

Consideriamo lo schema relazionale:

$$\text{TRENO}(\underline{\text{numero}}, \text{partOra}, \text{partMinuto}, \text{cat}, \text{dest}, \text{arrOra}, \text{arrMin})$$

$$\text{FERMATA}(\underline{\text{numTreno}, \text{ora}}, \text{min})$$

Si richiede di produrre la lista di tutti i treni non regionali che partono alle 12:00 o dopo le 12:00 e prima delle 16:00

$$\boldsymbol{\sigma_{\text{partOra} \geq 12 \ \wedge \ \text{partOra} < 16 \ \wedge \text{cat} \neq \text{'regionale'}}(\text{TRENO})}$$

Esempio: operatore di proiezione

Consideriamo lo schema relazionale:

$$\text{TRENO}(\underline{\text{numero}}, \text{partOra}, \text{partMinuto}, \text{cat}, \text{dest}, \text{arrOra}, \text{arrMin})$$

$$\text{FERMATA}(\underline{\text{numTreno}, \text{ora}}, \text{min})$$

Si richiede di restituire il numero e l'ora di partenza de treni Frecccia Rossa (cat = 'FR') con destinazione 'Milano Centrale'

$$\boldsymbol{\Pi_{\text{numero,partOra}}(\rho_{\text{cat} \ = \ \text{FR} \ \wedge \ \text{dest} \ = \ \text{'MilanoCentrale'}}(\text{TRENO}))}$$

6.2 Calcolo relazionale

7

Esercizi

7.1 Progettazione concettuale

7.1.1 Esercizio 1 - scheda

Esempio: esercizio 1

Progettare una base di dati che contenga le informazioni che descrivono la carriera di uno studente del corso di laurea triennale in informatica dal momento dell'immatricolazione no alla laurea.
In particolare, il sistema registra tutti gli esami sostenuti dagli studenti durante la loro carriera.

Per ogni **studente** si memorizzano: nome, cognome, data di nascita, matricola (univoca), data di immatricolazione e data di laurea.

Per ogni **insegnamento** erogato si memorizzano: denominazione, anno accademico di erogazione, anno di corso (I, II o III) e docente.

Si vuole tener traccia nella base di dati per ogni studente:
- ▷ *insegnamenti frequentati*: indicando percentuale di presenza (> 0) di uno studente alle lezioni
- ▷ *esami sostenuti*: indicando data dell'esame e voto finale

Si vuole, inoltre, poter calcolare quanti studenti hanno frequentato un insegnamento.

Questa soluzione presenta diverse problematiche:

- ▷ **Utente è legato a Esame** (Utente esiste se esiste anche Esame).
  Per questo motivo sarebbe consono creare un'entità Utente contente nome, cognome, matricola ecc.
- ▷ **cosa identifica nello specifico l'entità Esame?** Un appello? Un esame completo? ecc.
- ▷ **dentro all'entità dovrebbero esserci oggetti appartenenti ad un'unica classe** (oggetti simili)

7.1.2 Esercizio 2 - scheda

Esempio: esercizio 2

Si vuole progettare la basi di dati di una banca.

La banca è suddivisa in **filiali** e di ogni filiale si conosce: nome, codice numerico (univoco), indirizzo e numero di clienti.

I clienti della banca sono memorizzati nella base di dati.
Per ogni **cliente** sono memorizzati: nome, cognome, codice fiscale, data di nascita e professione.

La base di dati contiene, inoltre, i dati sui conti correnti aperti presso le filiali della banca.
Ogni conto corrente viene aperto presso una e una sola liale e ha una numerazione univoca nella liale.
Ogni cliente può aprire più conti (anche presso la stessa liale) e un conto può avere più intestatari.

Si vuole mantenere traccia nella base di dati di tutte le operazioni (movimenti) eseguite sui conti correnti, indicando per ogni movimento, un numero progressivo univoco per conto corrente, la data, il tipo di operazione e il valore.

Per ogni conto corrente si deve produrre il saldo.
Si supponga che un cliente possa aprire conti diversi in più filiali della banca.

7.1.3 Esercizio 3 - scheda

7.1.4 Esercizio 4 - scheda

7.2 Progettazione relazionale

7.2.1